

On The Books (Backend API)

On The Books is an event organizer's command center for managing supplier bookings across multiple events. This project is the backend API that powers it — a Node, Express, and MongoDB service that replaces the localStorage persistence used in the React front end with a real database, real user accounts, and a real authorization layer.

The API is built around four resources. A **User** owns many **Events**. A shared pool of **Suppliers** is available to every organizer, reflecting how the same vendors circulate among producers in the industry. An **Arrangement** is the junction between an event and a supplier, and it holds what is true about that supplier *at that event* — the negotiated budget, the booking status, and any notes. That separation is the core design decision: a supplier's name and category belong to the supplier, but "quoted ₱45,000, status declined" belongs to the arrangement.

All routes except registration and login require a JSON Web Token, sent as a Bearer token in the Authorization header. Events are scoped to their owner, so one organizer cannot read or modify another's events. Deletion is soft on Users, Events, and Suppliers — a flag is flipped rather than the record removed — and every read endpoint filters on that flag. Arrangements are the exception and are hard-deleted, because removing a supplier from an event is a correction rather than something a user would need to recover.

The API also exposes an AI recommendation endpoint, scoped to a single event. It reads that event's existing arrangements before responding, which lets it suggest replacements for suppliers who have declined and flag categories the event has not yet covered. Nothing it returns is persisted.

[Trello board link](#)

FEATURES

High

- Authentication and authorization
 - Register a new account with a username and password; passwords are hashed with Bcrypt before storage, never stored in plain text
 - Log in to receive a JSON Web Token
 - All routes other than register and login require a valid Bearer token
 - Requests with a missing, malformed, or expired token are rejected with 401
 - Events are scoped to their owner — a user cannot read or modify events belonging to another user
- User account
 - Retrieve the authenticated user's own profile
 - Soft-delete the account by flipping isActive
 - The route is /users/me rather than /users/:id, because the token already identifies the caller
- Event management
 - Create, read, and update events (name, date, location)
 - List all of the authenticated user's active events

- Retrieve a single event by ID
 - Soft delete by flipping `isActive`; inactive events are excluded from all reads
- Supplier pool management
 - Create, read, and update suppliers (name, category, contact)
 - Suppliers are a shared pool with no owner — every organizer sees and can use the same records
 - Category is constrained to a fixed set of twelve values, never free text
 - Soft delete by flipping `isActive`; inactive suppliers are excluded from all reads
- Arrangement management
 - Create, read, update, and delete arrangements within an event (budget, status, notes)
 - Status is constrained to four values: contacted, quoted, booked, declined
 - Arrangements are hard-deleted, giving the API one true DELETE verb
 - Reads exclude arrangements whose supplier has been deactivated
- AI supplier recommendation
 - Given an event, return suggested suppliers from the pool
 - The endpoint reads the event's existing arrangements first, so it can suggest replacements for declined suppliers and flag categories not yet covered
 - Results are returned to the client and not persisted — no new collection, no change to the schema
- Core technical
 - Express server with `dotenv`, `cors`, and `helmet`
 - MongoDB via `Mongoose`, with a schema per entity and referenced relationships
 - Secrets held in environment variables, never committed
 - Centralized error handling with correct status codes: 201 on create, 200 on read and update, 204 on delete, 400 on validation failure, 401 on auth failure, 404 on missing resource
 - Versioned, plural, hierarchically nested routes under `/api/v1/`
 - README documenting setup, environment variables, and every endpoint
 - Deployed to a hosting service (Render) with environment variables configured there
 - `Bcrypt` for password hashing and `JWT` for token management

Medium

- Filtering
 - GET `/suppliers` accepts an optional category query parameter to narrow results to a single category
- Populated reads
 - GET `/events/:eventId/arrangements` populates the referenced supplier, returning name, category, and contact alongside the arrangement's own budget, status, and notes, so the client can render a complete card in a single request

Low / Nice-to-have

- Jest test suite with 60%+ coverage
- Swagger API documentation
- Blog post documenting the build

USER STORIES

1. Register and log in (*High*)

As an event organizer, I want to register an account and log in, so that my events are private to me and persist across devices.

- a. Posting a username and password to the register endpoint creates an account and returns 201.
- b. The password is hashed before it is stored; the plain-text password is never written to the database or returned in any response.
- c. Registering with a username that already exists is rejected with 409 and a clear message.
- d. Posting valid credentials to the login endpoint returns 200 and a JSON Web Token.
- e. Posting invalid credentials returns 401 without revealing whether the username or the password was wrong.

2. Access protected resources (*High*)

As an event organizer, I want my data protected behind my login, so that no other user can read or change it.

- a. Every endpoint except register and login rejects a request with no Authorization header, returning 401.
- b. A malformed or expired token is also rejected with 401.
- c. A valid Bearer token identifies the caller, and the request proceeds without the client having to send a user ID.
- d. Requesting an event owned by a different user returns 404 rather than 403, so the API does not confirm that the event exists.

3. Manage my account (*High*)

As an event organizer, I want to view and deactivate my own account, so that I control my presence in the system.

- a. A GET to /users/me returns the authenticated user's profile without the password field.
- b. A PUT to /users/me setting isActive to false deactivates the account and returns 200.
- c. A deactivated user's token no longer grants access to protected routes.

4. Manage events (*High*)

As an event organizer, I want to create, view, update, and deactivate events, so that I have a dedicated space to track suppliers for each one.

- a. Posting an event with a name creates it, assigns the authenticated user as owner, and returns 201 with the created record.
- b. Posting an event without a name is rejected with 400.
- c. A GET to the events collection returns only the authenticated user's events where isActive is true.
- d. A GET to a single event by ID returns that event, or 404 if it does not exist, is inactive, or belongs to another user.
- e. A PUT updates the event's fields and returns 200.
- f. A PUT setting isActive to false removes the event from all subsequent reads while leaving the record in the database.

5. Manage the supplier pool (*High*)

As an event organizer, I want to add suppliers to a shared pool, so that vendors I have worked with are available to reuse across my events.

- a. Posting a supplier with a name and a valid category creates it and returns 201.

- b. A category outside the twelve permitted values is rejected with 400.
 - c. A GET to the suppliers collection returns all suppliers where isActive is true, regardless of who created them.
 - d. A GET with a category query parameter returns only suppliers in that category.
 - e. A PUT updates a supplier's details and returns 200.
 - f. A PUT setting isActive to false removes the supplier from all subsequent reads.
6. **Arrange suppliers for an event** (*High*)
- As an event organizer, I want to attach suppliers to a specific event with their own budget and status, so that I can track what each vendor costs and where each booking stands for that event.
- a. Posting an arrangement under an event, referencing a supplier and carrying a budget, status, and optional notes, creates it and returns 201.
 - b. A status outside the four permitted values — contacted, quoted, booked, declined — is rejected with 400.
 - c. A GET to an event's arrangements returns each arrangement with its supplier's name, category, and contact populated alongside the arrangement's own fields.
 - d. Arrangements whose supplier has been deactivated are excluded from that response.
 - e. A PUT updates an arrangement's budget, status, or notes and returns 200.
 - f. A DELETE removes the arrangement permanently and returns 204.
 - g. The same supplier can appear in arrangements under several different events, each with its own budget and status.
7. **Get AI supplier recommendations** (*High*)
- As an event organizer, I want the system to suggest suppliers for an event, so that I can fill gaps in my shortlist without searching the whole pool myself.
- a. Posting to an event's recommendations endpoint returns suggested suppliers drawn from the active pool.
 - b. The response accounts for the event's existing arrangements — it does not suggest suppliers already booked for that event.
 - c. Categories the event has no arrangement for are surfaced as gaps.
 - d. Nothing from the response is written to the database; a subsequent read of the event's arrangements is unchanged.
 - e. A failure from the AI provider returns a clear error rather than an empty success response.
8. **Filter suppliers by category** (*Medium*)
- As an event organizer, I want to narrow the supplier pool to a single category, so that I can find candidates for a specific need without scrolling the whole list.
- a. A GET to the suppliers collection with no query parameter returns all active suppliers.
 - b. A GET with ?category=catering returns only active suppliers in that category.
 - c. A category value outside the twelve permitted ones returns an empty array rather than an error.
 - d. Inactive suppliers are excluded whether or not the parameter is present.

ENDPOINTS

#	Action	URL	Verb	CRUD	Description	Status codes
1	Register user	/api/v1/auth/register	POST	Create	Creates a user with a hashed password.	201 · 400 · 409

#	Action	URL	Verb	CRUD	Description	Status codes
2	Log in user	/api/v1/auth/login	POST	Read	Verifies credentials and returns a JWT.	200 · 400 · 401
3	Get current user	/api/v1/users/me	GET	Read	Returns the logged-in user, identified from the token.	200 · 401
4	Deactivate user	/api/v1/users/me	PUT	Update	Soft delete — sets isActive to false.	200 · 401
5	List events	/api/v1/events	GET	Read	Returns all active events owned by the logged-in user.	200 · 401
6	Create event	/api/v1/events	POST	Create	Creates an event owned by the logged-in user.	201 · 400 · 401
7	Get one event	/api/v1/events/:id	GET	Read	Returns a single active event.	200 · 401 · 404
8	Update / deactivate event	/api/v1/events/:id	PUT	Update	Edits an event, or soft deletes it by setting isActive to false.	200 · 400 · 401 · 404
9	Recommend suppliers (AI)	/api/v1/events/:eventId/recommendations	POST	Read	Takes a plain-language description and returns AI-recommended suppliers, informed by the event's existing arrangements. Nothing is saved.	200 · 400 · 401 · 404
10	List arrangements for an event	/api/v1/events/:eventId/arrangements	GET	Read	Returns the event's arrangements with supplier details attached. Excludes arrangements whose supplier is inactive.	200 · 401 · 404
11	Add supplier to event	/api/v1/events/:eventId/arrangements	POST	Create	Creates an arrangement linking a supplier to the event.	201 · 400 · 401 · 404
12	Update arrangement	/api/v1/events/:eventId/arrangements/:id	PUT	Update	Edits the status, budget, or notes for that supplier at that event.	200 · 400 · 401 · 404
13	Remove supplier from event	/api/v1/events/:eventId/arrangements/:id	DELETE	Delete	Permanently removes the arrangement. The supplier record itself is untouched.	204 · 401 · 404
14	List suppliers	/api/v1/suppliers	GET	Read	Returns all active suppliers from the shared pool. Optional query parameter ?category= filters by category.	200 · 401
15	Create supplier	/api/v1/suppliers	POST	Create	Adds a supplier to the shared pool.	201 · 400 · 401

#	Action	URL	Verb	CRUD	Description	Status codes
16	Update / deactivate supplier	/api/v1/suppliers/:id	PUT	Update	Edits a supplier, or soft deletes it by setting isActive to false. Affects all users and all events.	200 · 400 · 401 · 404

Notes

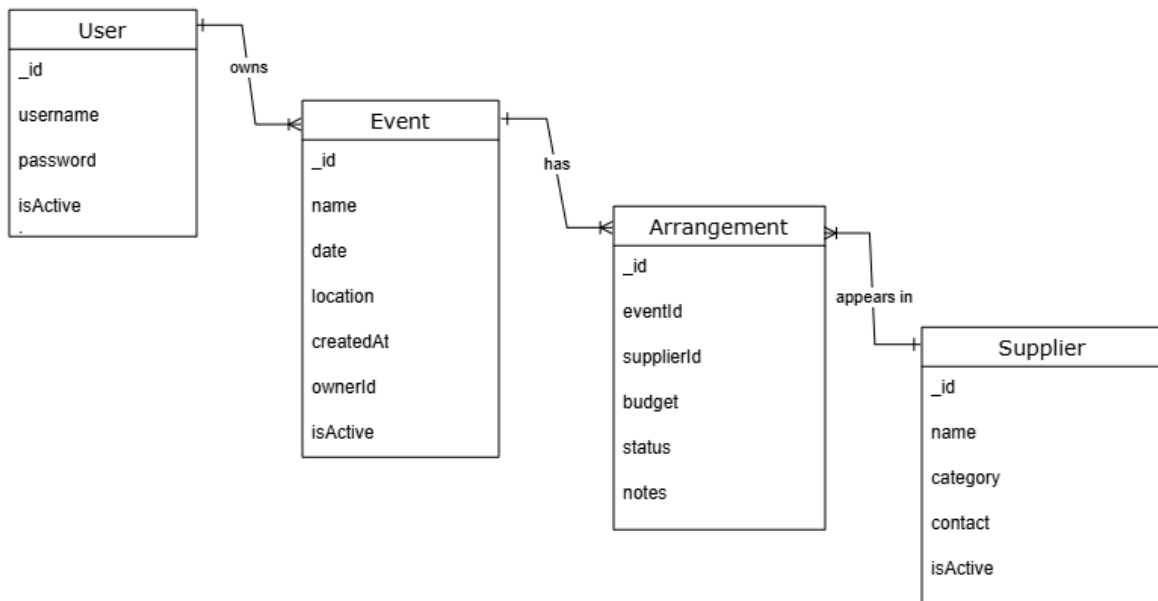
Endpoints 2 and 9 are marked Read but use POST. Both send a payload in the request body and neither creates a stored record; GET requests cannot carry a body.

Endpoint 13 is the only hard delete. Arrangement has no isActive field by design — removing a supplier from an event's shortlist is a correction, not something that needs recovery.

Suppliers are a shared pool with no ownerId, so endpoint 16 affects every user. This is a deliberate tradeoff: in the events industry the same suppliers circulate among producers, so whoever learns of a change first updates it for everyone.

ERD

Entity Relationship Diagram (ERD) for On the Books



UML Diagram for On the Books

